MDPI

Article

A Licence-Aware, Open-Source Pipeline for a Bilingual English–Turkish Aviation Corpus and Grounded Question-Answering Benchmark

Göksu Anıl Kızıldoğan [1,2]* and Melih Cemal Kuşhan [2]

[1] Analythinx; goksu.kizildogan@analythinx.com
[2] Eskişehir Osmangazi University; erzesk@ogu.edu.tr
* Correspondence: goksu.kizildogan@analythinx.com

Abstract

Large language models (LLMs) are increasingly used as retrieval-augmented assistants in aviation, yet no public question-answering (QA) resource combines English and Turkish aviation text for retrieval-augmented generation (RAG), fine-tuning, or evaluation. We present a reproducible, licence-aware, open-source pipeline and the resulting bilingual English–Turkish civil-aviation corpus and grounded QA benchmark. The pipeline acquires revision-pinned English and Turkish Wikipedia articles, EASA certification specifications, and Turkish and US authority documents. Every source and document carries an explicit rights state that an automatic audit enforces up to a fail-closed public release, and lightweight source-specific parsers turn spreadsheets, bookmarked PDFs, and regulations into provenance-linked text. QA pairs are generated by an open-weight LLM served with vLLM on a single consumer-grade RTX 3090 GPU, using an engineered prompt with schema-constrained outputs, and every answer is tied to character-offset evidence reconstructed by the pipeline. The benchmark covers five question types, cross-lingual items, and questions that are unanswerable by construction, verified against lexical and dense retrieval. Two independent open-weight LLM reviewers assess quality. The released benchmark contains 50 QA pairs over 3,310 passages from 1,004 documents; the reviewers judge 82% of the sampled items correct and grounded (Cohen's $\kappa = 0.32$). All code, prompts, and schemas are openly released.

Keywords: aviation; bilingual dataset; Turkish; question answering; retrieval-augmented generation; synthetic data generation; LLM-as-a-reviewer; data licensing

1. Introduction

Aviation is a document-heavy, safety-critical, and strongly regulated domain. Pilots, maintenance engineers, airport operators, and regulators work with certification specifications, advisory circulars, national regulations, traffic statistics, and a large body of encyclopaedic background knowledge. These texts are increasingly accessed through LLM assistants, most often in a retrieval-augmented generation (RAG) setting [1], where a retriever selects passages and a generator composes a grounded answer. Building and evaluating such systems requires domain QA data that is tied to evidence, covers answerable and unanswerable questions, and is legally redistributable.

For Turkish, a morphologically rich language with comparatively few resources, this need is acute. Türkiye has one of the world's largest aviation markets, and its national

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Aerospace* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

carrier ranks among the largest airlines by both passenger volume and fleet size. In practice, Turkish aviation professionals work in both languages: regulations and statistics are published in Turkish by national authorities such as SHGM and DHMİ, while type certification material, manufacturer documentation, and most technical literature are in English. A realistic assistant must therefore answer Turkish questions from English evidence and the reverse. In this paper we present a licence-aware, open-source pipeline that builds a bilingual English–Turkish aviation corpus and, from it, a grounded question-answering benchmark for RAG, fine-tuning, and evaluation.

Extractive QA was established by SQuAD [2], which pairs questions with answer spans in Wikipedia passages. SQuAD 2.0 [3] added adversarially written unanswerable questions so that systems must learn to abstain, but these questions are written against a single paragraph rather than a retrieval corpus. Cross-lingual benchmarks such as XQuAD [4] and MLQA [5], and the typologically diverse TyDi QA [6], extended this setting to other languages. Turkish appears in XQuAD, but only as professional translations of general-domain English paragraphs. Dedicated Turkish resources also target other domains: THQuAD [7] provides reading-comprehension pairs on Turkish-Islamic science history and Ottoman history, TurkishMMLU [8] evaluates multiple-choice knowledge from the Turkish high-school curriculum, and RAGTurk [9] benchmarks the stages of a RAG pipeline on Turkish Wikipedia and web text. None of these covers aviation, and none pairs Turkish questions with English evidence or the reverse.

Aviation NLP, in turn, is almost entirely English. Domain-adapted encoders such as Aviation-BERT [10] and aeroBERT [11] are trained on English safety narratives and certification requirements. AeroQA [12] provides multi-hop questions over an aviation knowledge graph built from NTSB reports, ATC-QA [13] derives QA pairs from ASRS incident reports, and Pre-Flight [14] is an expert-authored multiple-choice benchmark on aviation operational knowledge. All of these resources are English-only, and most are not designed as passage-grounded RAG benchmarks with released, licence-annotated retrieval corpora.

Two broader lines of work bear on how such a benchmark can be built. First, LLMs are widely used to synthesise training and evaluation data, for example instruction data [15] and queries for dense-retriever training [16,17], and RAG evaluation frameworks such as RAGAS [18] rely on LLM judgements. Using LLMs as judges is effective but subject to known biases [19], so a single unconstrained generator or judge cannot be trusted on its own. Second, datasheets [20] and large-scale audits of dataset licensing and provenance [21] show that licence information is often missing or wrong in widely used datasets, and is usually recorded only as a statement in documentation. To the best of our knowledge, no public dataset provides mixed English–Turkish, evidence-grounded aviation QA pairs, and none treats licence compliance as a checked property of the release.

Closing this gap raises problems beyond "asking an LLM for questions". First, the *rights* of source documents vary widely. Publicly accessible does not mean redistributable, and official Turkish sources often have unclear terms of reuse. Second, authoritative sources come in heterogeneous formats (spreadsheets with merged multi-level headers, bookmarked PDFs, regulations with numbered articles) that general-purpose converters handle poorly. Third, LLM-generated questions are frequently ambiguous outside their source passage ("When was the aircraft repainted?"), leak the answer, or paraphrase the answer in ways that cannot be scored. Fourth, a RAG benchmark needs questions that the corpus *cannot* answer, to measure abstention [3]. Finally, many research groups do not have data-centre hardware, so the pipeline should run on a single consumer GPU with open-weight models.

This paper addresses these gaps and problems with an end-to-end, file-based pipeline and the bilingual benchmark it produces. Our contributions are:

1. **A mixed English–Turkish aviation corpus and QA benchmark.** The corpus combines English and Turkish sources of different authority levels (community references, a supranational regulator, and national authorities) and is released with licence annotations as a retrieval corpus. The QA set contains monolingual and cross-lingual pairs in five question types, so Turkish questions are answered from English evidence and the reverse. It keeps the extractive contract for closed questions and adds explanatory questions with rubrics (Section 2).
2. **Licence-aware curation with automatic audit checks.** Each source and document carries a rights state (*open*, *manifest-only*, or *blocked*) with separate permissions for source bytes, derived text, and derived QA. A registry audit runs in continuous integration, and a fail-closed packager guarantees that restricted content never reaches passages, QA, or the public release. Licence compliance is therefore a tested property of the release rather than a statement in documentation (Section 2.3).
3. **Simple, deterministic parsers for authority documents.** Small, versioned extraction profiles handle DHMİ statistics workbooks, EASA certification specifications, SHGM regulations, and FAA advisory circulars, where general-purpose converters fail, and emit structured, provenance-linked artefacts (Section 3.1).
4. **A configurable English–Turkish ratio.** The corpus language mix is controlled by per-source caps and reported against a configurable reference ratio. The question-language split, answerability share, question-type mix, and cross-lingual share are set by a deterministic quota planner (Section 4.1).
5. **Prompt engineering for grounded generation and review.** We describe the generation prompt, which went through several revisions. It enforces self-contained questions, verbatim extractive answers, explicit rejection, and schema-constrained output, which removes ambiguous, leaking, and unscorable questions. The model never writes the evidence: the pipeline reconstructs it from stored character offsets, and every output is validated deterministically before any LLM review. We also describe a review prompt that scores four quality dimensions independently (Sections 4.2 and 5).
6. **Questions that are unanswerable by construction.** A controlled share of the questions is produced by deterministic entity/date/number substitution in answerable questions. Unlike SQuAD 2.0, these questions are defined relative to the frozen corpus rather than a single paragraph, and are verified to be unsupported by both lexical and dense retrieval (Section 4.3).
7. **Two independent AI reviewers, extensible to many AI or human reviewers.** Rather than trusting a single LLM judge, a stratified sample is double-reviewed by two open-weight LLMs of different sizes. Agreement is measured with Cohen's κ, and the reviewer-agnostic record format lets additional LLMs or human annotators take any slot (Section 5).
8. **A fully open-source stack that runs on one consumer GPU and scales through vLLM.** All models are open-weight and pinned to immutable revisions. They are served through vLLM's OpenAI-compatible server [22], so the same client runs on a single NVIDIA RTX 3090 (24 GiB) or on a multi-GPU deployment (Section 6).

The remainder of the paper is organised as follows. Sections 2–6 describe the sources and the rights model, corpus construction, QA generation, validation and review, and the serving infrastructure. Section 7 reports corpus and benchmark statistics, review agreement, and retrieval baselines. Section 8 discusses limitations and concludes with future work.

2. Sources and Licence-Aware Curation

2.1. Scope and taxonomy

Version 1 is a frozen snapshot of *civil* aviation knowledge (snapshot date 2026-07-31). Military and space material is excluded unless it appears as context in a civil regulator's document. Each document is assigned to one or more of eight topics: (1) regulation, standards, licensing, and passenger rights; (2) airlines, operators, fleets, and aviation economics; (3) airports, heliports, runways, navigation aids, and codes; (4) aircraft, airworthiness, certification, and manufacturers; (5) engines, propulsion, components, and maintenance; (6) operations, air traffic management, meteorology, and training; (7) safety, accidents, incidents, and human factors; and (8) general aviation, rotorcraft, balloons, and unmanned aircraft. Personal data (aircraft owners, airmen, passengers, victims, employees, addresses) is deliberately not curated. Time-sensitive claims about fleets, airports, and companies must carry an `as_of` date with field-level provenance.

2.2. Mixed English and Turkish sources

Sources are declared in a single registry file. Each entry specifies the publisher, source family, authority level, languages, topics, expected MIME types, acquisition adapter, extraction profile, and rights. Table 1 lists the sources enabled for this release. English material comes from English Wikipedia aviation categories, EASA certification specifications, OurAirports, and FAA advisory circulars. Turkish material comes from Turkish Wikipedia (Vikipedi) aviation categories and from two national authorities: DHMİ (airport traffic statistics) and SHGM (civil aviation regulations). The sources differ in authority level (community reference, supranational authority, national authority) and in native format (HTML, CSV, XLSX, PDF), which is reflected in the sampling diagnostics.

Table 1. Enabled sources, their languages, native formats, adapters, and rights states. *Manifest-only* sources are processed internally but only their metadata, checksums, and fetch recipes are released.

Source (publisher)	Lang.	Format	Adapter	Rights
English Wikipedia, aviation categories (400 pages max.)	EN	HTML	`mediawiki_api`	open (CC BY-SA 4.0)
Turkish Wikipedia, aviation categories (600 pages max.)	TR	HTML	`mediawiki_api`	open (CC BY-SA 4.0)
EASA Easy Access Rules, CS-E	EN	PDF	`local_glob`	open (EASA reuse)
OurAirports	EN	CSV	`direct`	open (public domain)
FAA Advisory Circulars	EN	PDF	`local_glob`	manifest-only
DHMİ airport statistics	TR	XLSX	`local_glob`	manifest-only
SHGM regulations	TR	PDF	`local_glob`	manifest-only

Registered but disabled or blocked: Wikipedia/Wikidata dumps, NTSB bulk data, national accident reports, proprietary aviation code products (blocked).

Wikipedia is accessed through bounded, polite queries to the MediaWiki Action API rather than full dumps. Category traversal is limited to one subcategory level, with page and byte caps per language (400 English and 600 Turkish pages, 512 MiB each). Every page is pinned to an immutable revision identifier, and the revision timestamp becomes the document's `as_of` date. Authority files (DHMİ, SHGM, EASA, FAA) are imported from a local directory through filename patterns. Each unique checksum becomes one immutable source version, so renaming a file never creates a duplicate. All acquired bytes are stored in a content-addressed store (`raw/sha256/`), and every acquisition event is recorded with request and final URLs, redirect chain, filtered headers, checksum, detected MIME type, and the fetch recipe. Network access is off by default and must be enabled explicitly.

2.3. Rights model and audit checks

Public availability is not evidence of permission to redistribute. Every source therefore declares one of three rights states:

open
: The source and its permitted derivatives may be packaged. The registry records three separate affirmative permissions: `release_source` (original bytes), `release_derived_text` (canonical text and passages), and `release_qa` (derived QA). It also records the licence identifier, licence and terms URLs, attribution text, and review date.

manifest-only
: Only public metadata, checksum, URL, and a fetch recipe may be released. The source may be parsed for internal diagnostics, but its derived text, passages, and QA must never be published. In this release, FAA, DHMİ, and SHGM are manifest-only because their reuse terms are either per-publication or unclear.

blocked
: The source is never acquired (for example, proprietary code products).

The rights state is copied from the source to every derived document and is enforced at four points. (i) A *registry audit*, run as a CI step, fails if any of the following holds: a blocked source is enabled; an enabled source uses an unimplemented adapter; an open source has inconsistent release permissions; a restricted source sets a release flag; the rights evidence (licence, terms URL, review date, attribution) is incomplete; a local file pattern could escape the project directory; or a MediaWiki source lacks its versioned extraction profile. (ii) Passage construction drops every document that is not open or lacks derived-text and QA permissions, and it reports how many documents were excluded for rights reasons. (iii) QA validation rejects any item whose evidence is not publicly releasable (`evidence_not_publicly_releasable`). (iv) The public packager is *fail-closed*: it re-verifies that every included document is open with affirmative permissions and that every evidence passage belongs to an included open document. It shards the output by licence family (for example, CC BY-SA content stays in a share-alike shard) and writes a checksum file. Restricted sources appear only in a scrubbed manifest. A dedicated end-to-end regression test runs the full pipeline on a small CC0 fixture portfolio and asserts that no restricted byte, text, passage, or QA pair crosses the public boundary.

3. Corpus Construction

The pipeline is linear and file-based (Table 2). Each stage reads the previous stage's JSON Lines (JSONL) output, validates it against a Pydantic schema, and writes its own versioned records, with Parquet mirrors. Every stage is a subcommand of a single command-line tool, is deterministic given its inputs and configuration, and can be rerun or resumed. The whole graph is also declared as a DVC pipeline [23], and an offline fixture run exercises every stage in CI.

Table 2. Pipeline stages. Every stage writes schema-validated JSONL (and Parquet) records. Rights enforcement points are marked with †.

Stage	Function	Main output
`rights audit`†	Registry consistency and rights-evidence checks	audit report
`fetch`	Revision-pinned acquisition into a content-addressed store	`SourceRecord`
`extract`	Source-specific or format-generic conversion to canonical text	`DocumentRecord`
`curate`	Deduplication, quality and privacy flags, language/topic diagnostics	accepted/rejected documents
`passages build`†	Token-bounded passages with exact offsets	`PassageRecord`
`qa build`†	Quota planning, LLM generation, validation, fill cycles	`QARecord`
`qa review-sample`, `llm-review`	Stratified double review by two LLM reviewers	review rows
`report`, `qa promote`	Gate evaluation and promotion of a passing run	pilot report
`evaluate`	BM25 and dense retrieval, answer metrics, LLM judge	evaluation reports
`package -public`†	Fail-closed, licence-sharded release with checksums	public package

3.1. Simple, source-specific document parsing

General-purpose PDF and spreadsheet converters lose much of the structure that makes authority documents useful: merged header cells, bookmark hierarchies, and numbered definitions. Instead of one complex converter, we use small, versioned *extraction profiles*. Each profile is 100–400 lines of deterministic code that targets one publisher's layout and writes both a readable canonical text and a structured JSON artefact that records provenance:

- **DHMİ statistics workbooks** (`dhmi_workbook_v1`). The parser expands merged cells, detects where the multi-row header ends (the first row with numeric data), builds hierarchical column names such as "2025 HAZİRAN SONU / İç Hat" (end of June 2025 / domestic), and classifies rows as data, total, or note rows. It preserves formulas next to their cached values and emits one readable Markdown table per worksheet.
- **EASA certification specifications** (`easa_toc_section_v1`). The parser reads the PDF bookmark tree (table of contents) and extracts the text between the bookmark coordinates of one eligible CS-E or AMC section. The section is chosen deterministically from a configured seed and the file checksum, so a large regulation contributes a reproducible, reviewable unit instead of hundreds of pages.
- **SHGM regulations** (`shgm_abbreviations_v1`). The parser locates the "Tanımlar ve Kısaltmalar" (definitions and abbreviations) article, segments the lettered entries, separates abbreviation aliases from their Turkish meanings (dropping English parenthetical expansions), and keeps page provenance.
- **FAA advisory circulars** (`faa_purpose_applicability_v1`). The parser skips the table of contents (up to the last "Contents" page) and extracts the real PURPOSE and APPLICABILITY sections.
- **Wikipedia/Vikipedi** (`mediawiki_article_v1`). The parser walks the article DOM in document order. It keeps headings, prose, captions, nested and definition lists, equations, and readable tables. It removes edit controls, citations, navigation and

authority-control templates, and non-content sections in both languages (“References”, “See also”, “Kaynakça”, “Ayrıca bakınız”, “Dış bağlantılar”).

Other inputs fall back to format-generic extractors: PDF through `pymupdf4llm`, with an automatic Tesseract OCR fallback (English+Turkish, 300 dpi) when OCR recovers more text than the embedded text layer, and DOCX, XLSX, HTML, and plain text. Every extraction records the extractor, profile, removed selectors, excluded sections, and block counts. It also raises automatic quality flags for detached leading headings, excessive duplicate lines, residual HTML boilerplate, oversized documents, and list-heavy documents.

Manual extraction check.

Automatic flags are only a proxy. After curation, the pipeline draws a 10% sample of accepted documents, stratified by language, native format, authority level, and topic set, and a human reviewer judges each canonical text in an interactive terminal tool as *usable* or *not usable* for grounded QA without manual cleanup. The criteria are: coherent source order, no dominating boilerplate, no material omissions or duplications, and readable tables. The release gate requires at least 95% usable extractions.

3.2. Curation

Curation accepts or rejects documents. It rejects exact duplicates, structural extraction failures, and oversized documents; it flags possible personal data; and it rejects current airline or fleet claims that lack an `as_of` date. Alternate formats and versions of the same work share a `variant_group_id` so that they do not inflate topic counts, and documents about the same real-world entity share an `entity_group_id`. Both identifiers are later used to build leakage-safe splits (Section 5.1).

Configurable English–Turkish ratio.

The corpus language mix is controlled at acquisition time by per-source caps (here 400 English and 600 Turkish Wikipedia pages plus the authority files). It is monitored at curation time against a configurable reference token share, 70% EN / 30% TR with a $\pm$5-point tolerance in this release. We deliberately make the corpus ratio *observational* rather than enforced: trimming valid Turkish documents to meet a quota would throw away scarce Turkish text. The report therefore shows the observed ratio for all accepted documents and for QA-eligible (open) documents separately. Two diversity constraints are real release gates, evaluated on QA-eligible documents only so that restricted sources cannot satisfy them: every topic must contribute at least 5% of tokens, and no source family may exceed 40%. The question-level language split is set independently by the QA planner (Section 4.1).

3.3. Passage construction

Accepted, QA-eligible documents are split into retrieval passages with a target of 600 tokens, a hard maximum of 900 tokens, and an overlap of 80 tokens. Tables are kept whole up to 1,200 tokens and marked as table passages. Token counts use the generator’s own tokenizer, loaded from a local copy whose commit and file checksums are verified before use, so passage budgets match the model’s context accounting exactly. Every passage stores its heading path (document title followed by section headings), page and table locations, and exact canonical character offsets; slicing the canonical text at those offsets must reproduce the passage byte for byte.

4. Grounded QA Generation

QA generation happens in isolated, checkpointed *runs*. A run freezes a snapshot of the passages, plans a fixed set of generation tasks, calls the generator, validates the output, and repeats *fill cycles* for any strata still below target. A run can be resumed with the same identifier after an interruption, and a run is copied to the benchmark location only after passing the promotion gates (Section 5).

4.1. Evidence anchors and quota planning

Evidence anchors.

Each passage is segmented into candidate *anchors*: contiguous spans of 8–160 tokens (a sentence group, a list, or a table fragment). For each anchor the planner stores exact offsets and decides which question types it can support, using lightweight bilingual cues. For example, copular or definitional patterns ("is", "refers to", "defined as"; Turkish "olarak", "tanımlanır", and the copular suffixes -dır/-dir/-dur/-dür) mark definition anchors; explicit years, dates, or clock times mark temporal anchors; and contrastive markers ("whereas", "compared", "than"; Turkish "oysa", "kıyasla", "daha", "sırasıyla") mark comparison anchors. Lists and tables yield list/table anchors with detected items. To limit topical concentration, a passage may be reused for at most four tasks.

Quota plan.

A deterministic planner turns a single target size N into exact per-stratum counts using largest-remainder rounding (Table 3). The question-language split is 50/50 EN/TR, 90% of items are answerable and 10% are corpus-unanswerable, and answerable items follow a fixed type mix. A further 10% of answerable items are *cross-lingual*: the question is written in the other language from its evidence (an English question over Turkish evidence, or the reverse). All of these proportions are single weights in the planner, so the English–Turkish balance of the question set can be changed without altering any other stage. The planner also reports capacity: if the corpus does not contain enough compatible anchors for a stratum, the build stops with an explicit capacity error instead of silently relaxing the plan.

Table 3. Quota plan used by the QA planner. Counts for a target of N items are derived by largest-remainder rounding; the realised counts for the released benchmark are reported in Table 8.

Dimension	Category	Share
Question language	English / Turkish	50% / 50%
Answerability	answerable / corpus-unanswerable	90% / 10%
Answerable type	factual	40%
	definition	30%
	list/table	15%
	comparison	10%
	temporal	5%
Cross-lingual	share of answerable items	10%
Data split	train / validation / test (by group)	70% / 15% / 15%

Answer contracts.

Every question type has a strict, machine-checkable answer contract. *Factual* and *temporal* items have exactly one answer item, which must be an exact contiguous substring of the evidence (temporal items must contain an explicit date, year, time, or duration). *List/table* items have one or more exact substrings in source order, joined with " ; " in the displayed answer. *Definition* and *comparison* items are explanatory: they have no answer items, but a grounded reference answer and a non-empty rubric of required points, which supports rubric-based or judge-based scoring. Closed answers are never translated, com-

puted, inferred, or reordered, so exact-match and span-based metrics remain meaningful in cross-lingual items.

4.2. Prompt engineering for generation

The generation prompt (`qa_generation_v3`) is a system prompt of roughly 3,240 tokens. We developed it iteratively by inspecting rejected and accepted items from pilot runs. Its main design decisions are:

1. **Anchor-only grounding; the model never writes evidence.** The user message is a JSON object with the question language, question type, passage language (which makes a task cross-lingual when it differs), the anchor text and identifier, the anchor's `section_path` (document title followed by headings), the detected list items, an optional required question term (Section 4.3), and the task's contract flags (exact substrings, source order, no translation, no calculation). The model returns only the question and answer fields. The evidence span is rebuilt by the pipeline from the stored offsets, so a hallucinated or paraphrased quote cannot enter the dataset.
2. **Self-contained questions.** Early pilot runs with prompt v2 produced questions that could only be understood next to their passage, for example "What fields and values are listed in the table?" or "When was the aircraft repainted again?". Such questions are useless in open retrieval, where the question is issued against thousands of documents. Prompt v3 therefore adds explicit rules: name the specific aircraft, airport, runway, airline, engine, organisation, regulation, or table the fact belongs to; take the name from the anchor or, failing that, from the `section_path`; keep the qualifiers that identify one value (year, variant, flight phase, unit, table column); resolve back-references ("the latter", "aynı", "yukarıdaki"); refer to the real-world subject rather than to "the passage" or "the table" ("metinde", "belirtilen"); and ask about table content, never about its layout. The prompt also states when *not* to add a name: general concepts ("What is angle of attack?") are already self-contained. It ends with a self-test: *would a reader who sees only the question know which entity and period it concerns?*
3. **Identifier preservation across languages.** In cross-lingual questions, common nouns in names may be translated ("Dalaman Havalimanı" → "Dalaman Airport"), but registrations, ICAO/IATA codes, runway designators, model designations, and regulation numbers are copied verbatim, and answer spans stay in the language of the evidence.
4. **Typed contracts and an explicit reject path.** Each type's contract is stated in the prompt, and the model may return `{"kind":"reject","reject_reason":...}` whenever the anchor cannot support the requested type without breaking a rule (for example, a temporal request over an anchor with no time expression, or a factual request whose subject cannot be identified). Rejection is cheaper and safer than a forced, ungrounded question.
5. **Bilingual few-shot examples with contrastive pairs.** For every type, the prompt gives English and Turkish examples, including cross-lingual cases, and "wrong versus correct" pairs that show the ambiguous question the model should avoid next to the self-contained version.
6. **Constrained decoding and deterministic sampling.** Each request carries a per-type JSON Schema as the response format, so vLLM's guided decoding guarantees parseable output. The model's reasoning ("thinking") mode is disabled, the temperature is 0.2, the maximum output length is 768 tokens, and every request uses a fixed seed derived from the run seed, the task index, and the retry attempt. The prompt

file's SHA-256 hash, the model and tokenizer revisions, and the container digest are stored in every QA record.

The length of the prompt has a practical cost: at about 3,240 tokens, it did not fit a 4,096-token context together with the anchor and the 768-token output budget. The context window was therefore raised to 8,192 tokens, which is still feasible on a 24 GiB GPU (Section 6).

4.3. Questions that are unanswerable by construction

A RAG system should abstain when the corpus does not contain the answer. Instead of asking an LLM to invent unanswerable questions, which tends to produce questions that are obviously off-topic or accidentally answerable, we derive them *deterministically* from accepted answerable questions:

1. During planning, an answerable *parent* task is given a `required_question_term`: an entity, year, or number that occurs in its anchor. The generator must include this exact term naturally in the question.
2. After the parent question passes validation, the term is replaced by a plausible substitute of the same kind (another entity name, year, or number) taken from a *different* donor document. The substitute must not occur in the parent evidence.
3. The mutated question is kept only if it is *not jointly supported* anywhere in the corpus. For the union of the BM25 top-20 passages, the dense (multilingual-E5 [24]) top-20 passages, and every passage that contains the replacement string, the validator checks whether any passage contains the replacement term together with at least two content words of the question's predicate. If one does, the item is rejected.

Each unanswerable item records its full mutation provenance (parent question and passage, source and replacement values, donor document, and checksums of the parent evidence and answer) but carries no answer, no evidence, and no parent provenance in the public record, so the parent answer cannot leak. These questions are unanswerable *with respect to the frozen corpus*, not necessarily false in the world. They test whether a RAG system abstains, rather than answering from a passage that is lexically similar but about a different entity or year.

Table 4. Examples from the released benchmark (answers abbreviated). *X-ling* marks cross-lingual items. The last row is a corpus-unanswerable mutation of the first row ("The Cavalese" → "OACA").

Lang.	Type	Question	Answer (evidence language)
EN	temporal	When did the Cavalese cable car crash occur?	3 February 1998 (EN)
EN	definition	How is crew scheduling defined?	The process of assigning crews to operate transportation systems, such as rail lines or airlines. (EN)
TR	definition	Balon nasıl tanımlanır?	Isıtılmış hava ya da havadan hafif bir gazla doldurulan, atmosferde uçabilen, genellikle sepetli hava taşıtı. (TR)
TR	list/table	Gazipaşa-Alanya Havalimanı'nın özet tablosunda hangi temel bilgiler yer almaktadır?	IATA: GZP, ICAO: LTFG; Sivil; İşletici: TAV; Açılış: 2010; … (TR)
EN, X-ling	factual	How many Boeing 777-300ER aircraft did Turkish Airlines lease from Jet Airways in December 2008?	3 adet Boeing 777-300ER (TR)
TR, X-ling	factual	Değişken pitch veya sabit hız pervaneli uçaklar, starter motor devresindeki hava yüklerini ve akımı azaltmak için hangi pitch konumunda çalıştırılır?	fine pitch (EN)
EN	unanswerable	When did OACA cable car crash occur?	— (no answer in corpus)

5. Validation and Review

Quality control has two layers. A deterministic validator first rejects every item that violates a checkable rule. A sample of the surviving items is then double-reviewed by two independent LLM reviewers. The promotion gate combines both layers.

5.1. Deterministic validation

Every generated record passes the following checks. Any failure rejects the record with a named reason, and the reasons are aggregated in the run report.

- **Schema and type contract.** The record parses against the QA schema (v1.1.0), and its answer fields satisfy the contract of its type (Section 4.1).
- **Exact evidence.** The evidence quote must equal the passage slice and the canonical-document slice at the stored offsets, byte for byte, and must match its SHA-256 checksum. The evidence must also be one of the planned anchors, and the anchor must be compatible with the item's type.
- **Extractiveness.** Every closed answer item is an exact substring of the evidence; list items keep source order; a temporal answer contains an explicit time expression.
- **No leakage or hallucinated detail.** The question may not contain a closed answer verbatim. Every number and capitalised name in the answer or rubric must appear in the evidence or in the question. Explanatory reference answers must share at least 50% of their content words with the evidence.
- **Language.** A lightweight bilingual detector checks that the question is written in its declared language.
- **Duplicates.** Exact duplicate questions are removed, and near-duplicates are detected with shingle-based signatures.
- **Rights.** All evidence must come from open documents with release permissions (Section 2.3).
- **Unanswerability.** Mutated items must pass the joint-support check of Section 4.3.

Valid records form a pool, from which a quota selector fills each stratum up to its planned count in task order. Surplus items are logged as overflow rather than discarded silently. Items are then assigned to train, validation, and test splits (70/15/15) by hashing a *split group*: the connected component of documents linked by shared variant groups, entity groups, or co-occurrence as evidence of the same question. All questions about the same entity or the same source work therefore fall into the same split, which prevents train–test leakage.

5.2. Two independent LLM reviewers

Stratified double-review sample.

A 50% sample of accepted items is drawn with deterministic, stratified sampling over question language, type, cross-lingual flag, answerability, topic, source family, and native format. Each sampled item is assigned to two reviewer slots, A and B, which gives 2× as many review rows as sampled items.

Reviewers.

Slot A is reviewed by the project's primary model (`RedHatAI/Qwen3.8-27B-INT4`, 27B parameters) and slot B by the fallback model (`cyankiwi/Qwen3.5-9B-AWQ-4bit`, 9B parameters). The two reviewers differ in size and quantisation scheme and receive no information about each other's judgements. The primary model is also the generator; we discuss the resulting self-preference risk in Section 8. Reviews are produced with the same constrained-decoding setup as generation.

Review prompt.

The review prompt (`qa_review`) asks for four *independent* binary judgements plus a short note:

clarity
: The question is unambiguous without the evidence, when issued against a corpus of thousands of aviation documents. General-concept questions ("Apron nasıl tanımlanır?") are clear without naming a document. Questions about one specific aircraft, airport, runway, or regulation must name it. Document- or layout-referential phrasing ("in the table", "belirtilen") is not clear. The `section_paths` context may identify the subject, but it does not make an unnamed subject clear.

correctness
: The answer (or, for unanswerable items, the absence of an answer) is consistent with the evidence.

evidence sufficiency
: The evidence alone justifies the answer; it is not vague, truncated, or unrelated.

language quality
: The question and answer are grammatical, natural, and in the declared question language.

The reviewer is told to judge only what is in front of it and not to use outside knowledge to second-guess a fact the evidence states. It is also told not to let a failure on one dimension influence the others. Subject names taken from the section path (for example, "XL426" for a sentence that only says "the aircraft") are explicitly allowed, so that reviewers do not penalise the self-contained questions that the generation prompt asks for.

Agreement and correctness metrics.

An item passes a review row if all four dimensions are true. Let n be the number of review rows and $\hat{p}$ the fraction of passing rows. We report $\hat{p}$ as the *correct-and-grounded rate* and a Wilson 95% interval for the error rate $1 - \hat{p}$ [25]. Agreement between the two reviewers on the binary pass/fail decision is measured with Cohen's kappa [26]:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = p_A\, p_B + (1 - p_A)(1 - p_B), \tag{1}$$

where p_o is the observed agreement on doubly reviewed items and p_A, p_B are the two reviewers' pass rates.

Promotion gates.

A run is promoted to the benchmark only if the accepted count equals the target, the quota plan is met exactly, the double review is complete (exactly $\lceil N \cdot r \rceil$ unique items and twice as many rows for review rate r), the correct-and-grounded rate is at least 0.70, and $\kappa \geq 0.30$. The thresholds are command-line parameters. The defaults (a 15% review sample, a correct-and-grounded rate of at least 0.95, and $\kappa \geq 0.70$) are intended for large releases; for the 50-item release reported here we review a larger share of the items and use the relaxed thresholds given above. Missing review data is reported as *not evaluated*, never as a pass.

Extensibility to many AI or human reviewers.

The two LLM reviewers write their judgements to `llm_review_A.jsonl` and `llm_review_B.jsonl`, which are merged into the same `human_reviews.jsonl` file and schema used for human annotators. Each row records a reviewer identifier (for LLMs, the model identifier and immutable revision). The gate computation does not depend on who filled in a row: human annotators can replace either slot, a third model can act as an adjudicator, or more slots can be added with a multi-rater agreement statistic such as Fleiss' kappa. The interactive terminal tool used for the manual extraction review could be adapted for human QA reviewers with little effort.

6. Open-Source Serving on a Single Consumer GPU

Everything open.

Every component of the pipeline is open. The code is released under the Apache 2.0 licence. The generator and both reviewers are open-weight models from the Qwen family [27], which we use as quantised community checkpoints. The dense retriever is the open multilingual-E5 model [24]. Serving uses vLLM [22] and Hugging Face Text Embeddings Inference (TEI), and document conversion uses open libraries (PyMuPDF/`pymupdf4llm`, Tesseract, `openpyxl`, `selectolax`). No proprietary API is called at any stage, so every generated item can be regenerated by third parties from pinned artefacts.

Pinned, auditable runtime.

Table 5 lists the runtime configuration. Models and tokenizers are pinned to immutable Hugging Face commit hashes, and the vLLM container is pinned by image digest rather than by a mutable tag. Before generating, the client runs a preflight check: it refuses to start unless the configuration contains these pins and the endpoint actually serves the configured model identifier. The generator configuration (model, revisions, container digest, prompt hash, sampling settings, and planner version) is embedded in every QA record.

Table 5. Serving configuration on a single NVIDIA RTX 3090 (24 GiB).

Setting	Value
Generator / reviewer A	`RedHatAI/Qwen3.8-27B-INT4` (27B, INT4)
Reviewer B / fallback generator	`cyankiwi/Qwen3.5-9B-AWQ-4bit` (9B, AWQ 4-bit)
Dense retriever	`intfloat/multilingual-e5-base` via TEI
Serving framework	vLLM 0.25.1, OpenAI-compatible API
Primary weights in GPU memory	17.32 GiB
`gpu-memory-utilization`	0.80 (19.2 GiB)
KV-cache capacity	25,941 tokens
Max. context length	8,192 tokens
Concurrent sequences	2
Co-located embedding server	≈3.35 GiB
Decoding	JSON-Schema guided, temperature 0.2, thinking disabled

Fitting a 27B model and an embedding server on one 24 GiB card.

Four-bit weight quantisation [28,29] reduces the 27B generator to 17.32 GiB. With the TEI embedding server (≈3.35 GiB) on the same GPU, a memory utilisation of 0.75 left less than 1 GiB for the KV cache, which was too little in practice. Raising it to 0.80 gives about 1.9 GiB of KV cache, or 25,941 tokens. That is enough for the 8,192-token context required by the generation prompt, at about three times the concurrency of a single full-length request. We cap concurrency at 2 sequences and run in eager mode (no CUDA graphs) to leave headroom. The 9B reviewer fits easily under the same settings. Because the RTX 3090 is a consumer card, the full pipeline, including generation, both reviews, and dense retrieval, can be reproduced on an ordinary workstation. Measured on this card, generating the benchmark took about 12 minutes (about 260 accepted items per hour) and the double review took about 13 minutes.

Scaling through vLLM.

The pipeline talks to models only through the OpenAI-compatible HTTP interface that vLLM exposes. Scaling therefore requires no code change: the same client can point to a vLLM deployment with tensor or pipeline parallelism across several GPUs, a larger `max-num-seqs` for higher throughput with PagedAttention's memory-efficient batching [22], or several replicas behind a load balancer. Larger models can replace the pinned checkpoints by changing one configuration file. Runs are checkpointed at the task level (raw responses, parsed records, rejections, and retries), so a long run on shared or preemptible hardware can resume exactly where it stopped. Increasing the Wikipedia page caps and the target N is enough to scale from the pilot to the planned full release of 5,000 documents and 20,000 QA pairs, and review sample sizes scale automatically with N.

7. Results

7.1. Corpus

The snapshot of 2026-07-31 contains 1,011 acquisition records and 1,011 extracted documents, with no fetch or extraction errors. Curation accepted 1,004 documents and rejected 7. Of the accepted documents, 1,002 are open and eligible for QA; the rest are manifest-only authority documents that were processed internally but excluded at the rights boundary. Table 6 breaks the corpus down by source. The accepted corpus contains 726,441 canonical tokens, of which 63.7% are English and 36.3% Turkish, compared with the 70/30 reference. The English share is 6.3 points below the reference and therefore just outside the ±5-point tolerance. The deviation follows from the configured page caps, which admit more Turkish (600) than English (400) Wikipedia pages; English pages are longer on average (about 1,150 vs. 440 tokens), which only partly offsets the difference. Because the language ratio is observational, the deviation is reported but does not block

the release. In the manual extraction check, 98.1% of 106 sampled documents were judged usable, which is above the 95% gate.

Table 6. Corpus composition by source after curation. Passages are built only from open, QA-eligible documents. The four OurAirports CSV files and the DHMİ workbook were rejected during curation (oversized documents and a missing as-of date for current airline claims, respectively).

Source	Lang.	Rights	Documents	Tokens	Passages
English Wikipedia	EN	open	400	461,959	1,452
Turkish Wikipedia	TR	open	598	263,463	1,853
EASA CS-E	EN	open	1	278	1
OurAirports	EN	open	0	0	0
Bundled CC0 fixtures	EN/TR	open	3	211	4
FAA AC	EN	manifest-only	1	400	0
DHMİ statistics	TR	manifest-only	0	0	0
SHGM regulation	TR	manifest-only	1	130	0
Total			1,004	726,441	3,310

The passage stage produced 3,310 passages (1,456 English, 1,854 Turkish), including 737 table passages, with a mean length of 432 tokens. Over QA-eligible documents, the topic token shares are 16.7% each for aircraft, airports, general aviation, operations, and safety, 10.6% for engines, 6.0% for airlines, and below 0.01% for regulation. Regulation therefore falls short of the 5% topic minimum, and the 40% source-family cap is exceeded. Both diversity gates are therefore not met in this release: the regulatory and authority sources that would supply this material are manifest-only (Section 8).

7.2. QA generation and validation

For a target of $N = 50$ items, the generator produced 66 candidate records over 6 fill cycles. Guided decoding yielded a JSON-schema success rate of 100% (gate: $\geq$99%). The record-construction success rate, i.e., the fraction of model responses that became a well-formed record (not a model reject and not a broken contract), was 58.9%. The validator rejected 16 records; Table 7 lists the rejection reasons at both stages. Most generation-stage rejections are deliberate model rejects, where the prompt instructs the generator to decline an anchor that cannot support the requested question type, rather than broken outputs. The near-duplicate rate among generated questions was 0%.

Table 7. Rejection reasons at the generation and validation stages.

Stage	Reason	Count
Generation	model reject (anchor cannot support type)	34
Generation	closed-answer contract broken (not exactly one item)	3
Generation	mutation parent failed	3
Validation	unanswerable item jointly supported by corpus	7
Validation	explanatory answer weakly grounded	5
Validation	unsupported name or number	3
Validation	invalid mutation replacement	2
Validation	question language mismatch	1
Validation	duplicate or near-duplicate question	0

The final benchmark contains 50 QA pairs: 25 English and 25 Turkish questions, 5 cross-lingual items, and 5 corpus-unanswerable items (Table 8).

Table 8. Composition of the released QA benchmark by question language, question type, and cross-lingual status. Mono.: question and evidence in the same language; X-ling: question in the other language from its evidence.

	English questions		Turkish questions		
Type	Mono.	X-ling	Mono.	X-ling	Total
Factual	8	1	8	1	18
Definition	6	1	6	1	14
List/table	3	0	3	1	7
Comparison	2	0	2	0	4
Temporal	1	0	1	0	2
Unanswerable	3	—	2	—	5
Total	25		25		50

7.3. LLM double review

The stratified sample contains 25 items and 50 review rows. Table 9 reports per-dimension pass rates for both reviewers and their agreement. Overall, 82% of review rows judged the item correct and grounded on all four dimensions (95% Wilson interval for the error rate: 9.8–30.8%), and the two reviewers agreed with Cohen's $\kappa = 0.32$ (gate: $\kappa \geq 0.30$). Both promotion gates are met. Most clarity failures concern questions that refer to "the list" or "the table" without naming the source, which is exactly the self-containment problem the generation prompt targets. With 25 sampled items, per-dimension figures are indicative only.

Table 9. Pass rates of the two LLM reviewers per dimension and inter-reviewer agreement (Cohen's κ). "All four" is the item-level acceptance criterion used by the promotion gate.

Dimension	Reviewer A (27B)	Reviewer B (9B)	κ
Clarity	92%	88%	−0.11
Correctness	92%	96%	−0.06
Evidence sufficiency	100%	96%	0.00
Language quality	88%	92%	0.78
All four	80%	84%	0.32

7.4. Retrieval baselines

Table 10 reports BM25 [30] ($k_1 = 1.2$, $b = 0.75$) and dense multilingual-E5 retrieval over all released passages, for answerable questions, broken down by question language and cross-lingual status. The two retrievers perform similarly overall (nDCG@10 of 0.735 and 0.741), but they differ by language. BM25 is strong on English questions (R@10 of 0.955) and weaker on Turkish (0.783), consistent with Turkish agglutinative morphology, where inflected forms in the question often do not match the surface forms in the evidence. The dense retriever reverses the pattern at the top of the ranking (MRR@10 of 0.768 on Turkish vs. 0.617 on English). On cross-lingual questions, BM25's MRR@10 drops from 0.723 to 0.440, whereas the dense retriever degrades less (0.703 to 0.620). BM25's cross-lingual recall stays relatively high because these questions often keep names, model designations, and technical terms (e.g., "Boeing 777-300ER", "fine pitch") that also occur verbatim in the evidence. The cross-lingual subset contains only five items, so these differences are indicative only.

Table 10. Retrieval baselines on answerable questions (R@k: recall at k; MRR and nDCG at 10). Language subsets are by question language and include cross-lingual items.

Retriever	Subset	n	R@5	R@10	R@20	MRR@10	nDCG@10
BM25	All	45	0.844	0.867	0.911	0.691	0.735
	EN questions	22	0.909	0.955	0.955	0.725	0.782
	TR questions	23	0.783	0.783	0.870	0.659	0.691
	Monolingual	40	0.850	0.875	0.925	0.723	0.761
	Cross-lingual	5	0.800	0.800	0.800	0.440	0.530
mE5-base	All	45	0.822	0.889	0.933	0.694	0.741
	EN questions	22	0.773	0.909	0.955	0.617	0.686
	TR questions	23	0.870	0.870	0.913	0.768	0.794
	Monolingual	40	0.850	0.900	0.950	0.703	0.751
	Cross-lingual	5	0.600	0.800	0.800	0.620	0.658

7.5. Released artefacts

The public package contains source and document metadata, canonical text, passages, and QA records as JSONL and Parquet (plus a QA CSV), sharded by licence family. It also contains JSON Schemas for all record types, a scrubbed manifest of restricted sources, and a checksum file for every artefact. Each QA record includes its exact evidence spans with passage and canonical offsets, question and evidence languages, cross-lingual label, answer items or reference answer and rubric, deterministic acceptable variants, split assignment, and the full generator configuration. The code, configurations, prompts, and schemas are available at https://github.com/gkizildogan/datacuration.

8. Discussion and Conclusions

8.1. Limitations

Rights versus coverage.

The rights model trades coverage for legal clarity. The Turkish national authority sources (DHMİ, SHGM) and the FAA advisory circulars have unclear or per-publication reuse terms, so they stay manifest-only. The parsers developed for them are therefore used for internal diagnostics and for users who rebuild the extension locally from the released fetch recipes, but these sources contribute no public QA. As a result, the source-family diversity gate is not met in this release. Obtaining explicit reuse permission from Turkish authorities is the most direct way to increase the share of authoritative Turkish text.

LLM reviewers are not human experts.

Two LLM reviewers make large-scale double review affordable on one GPU, but their agreement measures consistency, not validity. Both reviewers come from the same model family, and reviewer A is also the generator, which may inflate pass rates through self-preference [19]. The reviewer-agnostic review format (Section 5) makes it easy to calibrate against human aviation experts on a subsample, or to replace reviewer A with a model from a different family. We regard this calibration as necessary before the benchmark is used for high-stakes model selection.

Synthetic unanswerable questions.

Deterministic substitution guarantees provable corpus-unanswerability, but some mutations are semantically odd (for example, a person's name substituted where an engine designation was expected). Such items are easier to reject than realistic near-miss questions. Type-aware donor selection, for example replacing an aircraft only with another aircraft, would make them harder and more realistic.

Heuristic components.

Anchor typing and question-language detection rely on lightweight bilingual lexical cues rather than trained classifiers. The extractive contract for closed questions makes scoring objective, but it can produce long list/table answers and excludes questions whose answers require light computation or aggregation. Cross-lingual items make up only 10% of answerable questions by design.

Frozen snapshot, not an operational source.

The benchmark is a frozen research snapshot. Fleet, airport, and regulatory facts carry `as_of` dates and describe their sources as of the snapshot date (2026-07-31). The data must not be used as an operational aeronautical information, navigation, safety, or regulatory source.

8.2. Conclusions and Future Work

We presented a licence-aware, fully open-source pipeline and the resulting bilingual English–Turkish civil-aviation corpus and grounded QA benchmark, to our knowledge the first public resource of its kind. The pipeline combines English and Turkish sources of different authority levels. It enforces per-source and per-document rights with automatic audits and a fail-closed release, and it parses authority documents such as DHMİ workbooks and EASA certification specifications with small, deterministic profiles. The English–Turkish balance of both the corpus and the question set is configurable. On the generation side, prompt engineering, schema-constrained decoding, and pipeline-reconstructed evidence produce self-contained, extractively grounded questions of five types, including cross-lingual questions and questions that are unanswerable by construction. Two independent open-weight LLM reviewers, which can be complemented or replaced by additional AI or human reviewers, gate the release. The whole workflow runs on a single consumer-grade RTX 3090 through vLLM and scales to larger deployments without code changes. The released benchmark contains 50 QA pairs, which the reviewers judge 82% correct and grounded ($\kappa = 0.32$). It supports evaluation of bilingual and cross-lingual retrieval, grounded generation, and abstention in aviation RAG systems.

We plan to scale the release to the planned 5,000 documents and 20,000 QA pairs, and to seek reuse permission from Turkish aviation authorities so that their documents can move from manifest-only to open. We will calibrate the LLM reviewers against human aviation experts and add reviewers from other model families. Finally, we will use the benchmark to fine-tune and evaluate bilingual aviation RAG systems end to end.

Author Contributions: Conceptualization, G.A.K.; methodology, G.A.K.; software, G.A.K.; validation, G.A.K.; formal analysis, G.A.K.; investigation, G.A.K.; resources, M.C.K.; data curation, G.A.K. and M.C.K.; writing—original draft preparation, G.A.K. and M.C.K.; writing—review and editing, G.A.K. and M.C.K.; visualization, G.A.K.; supervision, M.C.K.; project administration, M.C.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The code, configurations, prompts, and schemas are available at https://github.com/gkizildogan/datacuration. The public package contains only sources whose licences permit redistribution; restricted sources are described by a scrubbed manifest with fetch recipes.

Acknowledgments: The authors thank Analythinx for its encouragement of this work. During the preparation of this manuscript/study, the author(s) used Claude (Claude application and Claude Opus 5.5 model) for the purposes of generating boilerplate parts of the code. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest. Göksu Anıl Kızıldoğan is employed by Analythinx; the company had no role in the design of the study, the analyses, the writing of the manuscript, or the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

AC	Advisory Circular
CI	Continuous integration
CS-E	Certification Specifications for Engines
DHMİ	Devlet Hava Meydanları İşletmesi
EASA	European Union Aviation Safety Agency
FAA	Federal Aviation Administration
GPU	Graphics processing unit
JSONL	JSON Lines
KV	Key–value (cache)
LLM	Large language model
MRR	Mean reciprocal rank
nDCG	Normalised discounted cumulative gain
OCR	Optical character recognition
QA	Question answering
RAG	Retrieval-augmented generation
SHGM	Sivil Havacılık Genel Müdürlüğü
TEI	Text Embeddings Inference

References

1. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the Advances in Neural Information Processing Systems, 2020, Vol. 33, pp. 9459–9474.
2. Rajpurkar, P.; Zhang, J.; Lopyrev, K.; Liang, P. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In Proceedings of the Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, 2016, pp. 2383–2392.
3. Rajpurkar, P.; Jia, R.; Liang, P. Know What You Don't Know: Unanswerable Questions for SQuAD. In Proceedings of the Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2018, pp. 784–789.
4. Artetxe, M.; Ruder, S.; Yogatama, D. On the Cross-lingual Transferability of Monolingual Representations. In Proceedings of the Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 4623–4637.
5. Lewis, P.; Oguz, B.; Rinott, R.; Riedel, S.; Schwenk, H. MLQA: Evaluating Cross-lingual Extractive Question Answering. In Proceedings of the Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 7315–7330.
6. Clark, J.H.; Choi, E.; Collins, M.; Garrette, D.; Kwiatkowski, T.; Nikolaev, V.; Palomaki, J. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics* **2020**, *8*, 454–470.
7. Soygazi, F.; Çiftçi, O.; Kök, U.; Cengiz, S. THQuAD: Turkish Historic Question Answering Dataset for Reading Comprehension. In Proceedings of the 2021 6th International Conference on Computer Science and Engineering (UBMK), 2021, pp. 215–220.
8. Yüksel, A.; Köksal, A.; Senel, L.K.; Korhonen, A.; Schütze, H. TurkishMMLU: Measuring Massive Multitask Language Understanding in Turkish. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 7035–7055.
9. Köse, S.K.; Baytekin, M.C.; Aktaş, B.; Görür, B.K.; Munis, E.A.; Yılmaz, D.; Kartal, M.Y.; Toraman, C. RAGTurk: Best Practices for Retrieval Augmented Generation in Turkish. In Proceedings of the Proceedings of the Second Workshop on Natural Language Processing for Turkic Languages (SIGTURK 2026), 2026, pp. 179–196.

10. Chandra, C.; Jing, X.; Bendarkar, M.V.; Sawant, K.; Elias, L.; Kirby, M.; Mavris, D.N. Aviation-BERT: A Preliminary Aviation-Specific Natural Language Model. In Proceedings of the AIAA AVIATION 2023 Forum, 2023. https://doi.org/10.2514/6.2023-3436.
11. Tikayat Ray, A.; Cole, B.F.; Pinon Fischer, O.J.; White, R.T.; Mavris, D.N. aeroBERT-Classifier: Classification of Aerospace Requirements Using BERT. *Aerospace* **2023**, *10*, 279.
12. Agarwal, A.; Gawade, S.; Azad, A.P.; Bhattacharyya, P. KITLM: Domain-Specific Knowledge InTegration into Language Models for Question Answering. In Proceedings of the Proceedings of the 20th International Conference on Natural Language Processing (ICON), 2023.
13. Shi, H.; Yu, Z. ATC-QA: A Benchmark for Evaluating LLMs on Aviation Safety Compliance. Springer Nature Singapore, book chapter, 2026. https://doi.org/10.1007/978-981-95-6366-1_34.
14. Brooker, A.; Hughes, T. Pre-Flight: A Benchmark for Evaluating Large Language Models on Aviation Operational Knowledge, 2026, [arXiv:cs.CL/2607.01829].
15. Wang, Y.; Kordi, Y.; Mishra, S.; Liu, A.; Smith, N.A.; Khashabi, D.; Hajishirzi, H. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In Proceedings of the Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 13484–13508.
16. Bonifacio, L.; Abonizio, H.; Fadaee, M.; Nogueira, R. InPars: Unsupervised Dataset Generation for Information Retrieval. In Proceedings of the Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2022, pp. 2387–2392.
17. Dai, Z.; Zhao, V.Y.; Ma, J.; Luan, Y.; Ni, J.; Lu, J.; Bakalov, A.; Guu, K.; Hall, K.B.; Chang, M.W. Promptagator: Few-shot Dense Retrieval From 8 Examples. In Proceedings of the International Conference on Learning Representations (ICLR), 2023.
18. Es, S.; James, J.; Espinosa-Anke, L.; Schockaert, S. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In Proceedings of the Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 2024, pp. 150–158.
19. Zheng, L.; Chiang, W.L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.P.; et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Proceedings of the Advances in Neural Information Processing Systems (Datasets and Benchmarks Track), 2023, Vol. 36.
20. Gebru, T.; Morgenstern, J.; Vecchione, B.; Vaughan, J.W.; Wallach, H.; Daumé III, H.; Crawford, K. Datasheets for Datasets. *Communications of the ACM* **2021**, *64*, 86–92.
21. Longpre, S.; Mahari, R.; Chen, A.; Obeng-Marnu, N.; Sileo, D.; Brannon, W.; Muennighoff, N.; Khazam, N.; Kabbara, J.; Perisetla, K.; et al. The Data Provenance Initiative: A Large Scale Audit of Dataset Licensing & Attribution in AI, 2023, [arXiv:cs.CL/2310.16787].
22. Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C.H.; Gonzalez, J.E.; Zhang, H.; Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), 2023, pp. 611–626.
23. Iterative, Inc.. DVC: Data Version Control. https://dvc.org, 2026. Accessed 2026-09-29.
24. Wang, L.; Yang, N.; Huang, X.; Yang, L.; Majumder, R.; Wei, F. Multilingual E5 Text Embeddings: A Technical Report, 2024, [arXiv:cs.CL/2402.05672].
25. Wilson, E.B. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association* **1927**, *22*, 209–212.
26. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* **1960**, *20*, 37–46.
27. Qwen Team. Qwen3 Technical Report, 2025, [arXiv:cs.CL/2505.09388].
28. Frantar, E.; Ashkboos, S.; Hoefler, T.; Alistarh, D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In Proceedings of the International Conference on Learning Representations (ICLR), 2023.
29. Lin, J.; Tang, J.; Tang, H.; Yang, S.; Chen, W.M.; Wang, W.C.; Xiao, G.; Dang, X.; Gan, C.; Han, S. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In Proceedings of the Proceedings of Machine Learning and Systems (MLSys), 2024, Vol. 6.
30. Robertson, S.; Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* **2009**, *3*, 333–389.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.